知乎 首发于 好好学习 切换模式 写文章

Python sys 模块详解

轩辕御龙
ZGNB

95 人赞同了该文章

Python sys 模块详解

1. 简介

“sys”即“system”，“系统”之意。该模块提供了一些接口，用于访问 Python 解释器自身使用和维护的变量，同时模块中还提供了一部分函数，可以与解释器进行比较深度的交互。

2. 常用功能

2.1 sys.argv

“argv”即“argument value”的简写，是一个列表对象，其中存储的是在命令行调用 Python 脚本时提供的“命令行参数”。

这个列表中的第一个参数是被调用的脚本名称，也就是说，调用 Python 解释器的“命令”（`python`）本身并没有被加入这个列表当中。这个地方要注意一下，因为这一点跟 C 程序的行为有所不同，C 程序读取命令行参数是从头开始的。

举例来说，在当前目录下新建一个 Python 文件 `sys_argv_example.py`，其内容为：

```
import sys

print("The list of command line arguments:\n", sys.argv)
```

在命令行运行该脚本：

```
$ python sys_argv_example.py
The list of command line arguments:
 ['example.py']
```

加上几个参数试试：

```
$ python sys_argv_example.py arg1 arg2 arg3
The list of command line arguments:
 ['example.py', 'arg1', 'arg2', 'arg3']
```

利用好这个属性，可以极大增强 Python 脚本的交互性。

2.2 sys.platform

在《第26天： Python 标准库之 os 模块详解》中，我们提到过“查看 `sys` 模块中的

▲ 赞同 95 ▼ 6 条评论 分享 喜欢 收藏 申请转载 ···

```
'win32'
```

在 Linux 上：

```
>>> sys.platform
'linux'
```

比较一下 `os.name` 的结果，不难发现， `sys.platform` 的信息更加准确。

2.3 sys.byteorder

“byteorder”即“字节序”，指的是在计算机内部存储数据时，数据的低位字节存储在存储空间中的高位还是低位。

“小端存储”时，数据的低位也存储在存储空间的低位地址中，此时 `sys.byteorder` 的值为“`little`”。如果不注意，在按地址顺序打印内容的时候，可能会把小端存储的内容打错。当前**大部分机器**都是使用的小端存储。

所以不出意外的话，你的机器上执行下述交互语句也应当跟我的结果一样：

```
>>> sys.byteorder
'little'
```

而另外还存在一种存储顺序是“大端存储”，即数据的高位字节存储在存储空间的低位地址上，此时 `sys.byteorder` 的值为“`big`”。

这种方式看起来好像很合理也很自然，因为我们一般在书面表示的时候都将低位地址写在左边，高位地址写在右边，大端存储的顺序就很符合人类的阅读习惯。但实际上对机器而言，内存地址并没有左右之分，所谓的“自然”其实并不存在。

抱歉我并没有使用大端存储的机器可以用作演示，因此只能说如果是大端存储的机器上运行 Python，输出结果应该像下面这样，也就是说下面这个示例并非我得到的真实运行结果，仅供参考：

```
>>> sys.byteorder
'big'
```

2.4 sys.executable

该属性是一个字符串，在正常情况下，其值是当前运行的 Python 解释器对应的可执行程序所在的绝对路径。

比如在 Windows 上使用 Anaconda 安装的 Python，该属性的值就是：

```
>>> sys.executable
'E:\\Anaconda\\Anaconda\\python.exe'
```

2.5 sys.modules

该属性是一个字典，包含的是各种已加载的模块的模块名到模块具体位置的映射。

关于其具体的值，由于内容过多，就不在此给出示例了，读者可以自行查看。

2.6 sys.builtin_module_names

该属性是一个字符串元组，其中的元素均为当前所使用的的 Python 解释器内置的模块名称。

注意区别 `sys.modules` 和 `sys.builtin_module_names` ——前者的关键字（keys）列出的是导入的模块名，而后者则是解释器内置的模块名。

其值示例如下：

```
>>> sys.builtin_module_names
('_abc', '_ast', '_bisect', '_blake2', '_codecs', '_codecs_cn', '_codecs_hk', '_codecs
```

2.7 sys.path

A list of strings that specifies the search path for modules. Initialized from the environment variable `PYTHONPATH`, plus an installation-dependent default.

该属性是一个由字符串组成的列表，其中各个元素表示的是 Python 搜索模块的路径；在程序启动期间被初始化。

其中第一个元素（也就是 `path[0]` ）的值是最初调用 Python 解释器的脚本所在的绝对路径；如果是在交互式环境下查看 `sys.path` 的值，就会得到一个空字符串。

命令行运行脚本（脚本代码见示例 `sys_path_example.py` ）：

```
$ python sys_path_example.py
The path[0] =  D:\justdopython\sys_example
```

交互式环境查看属性第一个元素：

```
>>> sys.path[0]
''
```

3. 进阶功能

3.1 sys.stdin

即 Python 的标准输入通道。通过改变这个属性为其他的类文件（file-like）对象，可以实现输入的重定向，也就是说可以用其他内容替换标准输入的内容。

所谓“标准输入”，实际上就是通过键盘输入的字符。

在示例（ `sys_stdin_example.py` ）中，我们尝试把这个属性的值改为一个打开的文件对象 `hello_python.txt` ，其中包含如下的内容：

```
Hello Python!
Just do Python, go~
```

```
$ python sys_stdin_example.py
Hello Python!
Just do Python, go~

Go, Go, GO!
```

3.2 sys.stdout

与上一个“标准输入”类似， `sys.stdout` 则是代表“标准输出”的属性。

通过将这个属性的值修改为某个文件对象，可以将本来要打印到屏幕上的内容写入文件。

比如运行示例程序 `sys_stdout_example.py` ，用来临时生成日志也是十分方便的：

```
import sys


# 以附加模式打开文件，若不存在则新建
with open("count_log.txt", 'a', encoding='utf-8') as f:
    sys.stdout = f
    for i in range(10):
        print("count = ", i)
```

3.3 sys.err

与前面两个属性类似，只不过该属性标识的是标准错误，通常也是定向到屏幕的，可以粗糙地认为是一个输出错误信息的特殊的标准输出流。由于性质类似，因此不做演示。

此外， `sys` 模块中还存在几个“私有”属性： `sys.__stdin__` ， `sys.__stdout__` ， `sys.__stderr__` 。这几个属性中保存的就是最初定向的“标准输入”、“标准输出”和“标准错误”流。在适当的时侯，我们可以借助这三个属性将 `sys.stdin` 、 `sys.stdout` 和 `sys.err` 恢复为初始值。

3.4 sys.getrecursionlimit() 和 sys.setrecursionlimit()

`sys.getrecursionlimit()` 和 `sys.setrecursionlimit()` 是成对的。前者可以获取 Python 的最大递归数目，后者则可以设置最大递归数目。因为初学阶段很少用到，因此只做了解。

3.5 sys.getrefcount()

在《第12天：Python 之引用》中我们其实已经用到过这个函数，其返回值是 Python 中某个对象被引用的次数。关于“引用”的知识可以回去看看这篇文章。

3.6 sys.getsizeof()

这个函数的作用与 C 语言中的 `sizeof` 运算符类似，返回的是作用对象所占用的字节数。

比如我们就可以看看一个整型对象 `1` 在内存中的大小：

```
>>> sys.getsizeof(1)
```

```
>>> sys.getsizeof(2**30-1)
28
>>> sys.getsizeof(2**30)
32
```

3.7 sys.int_info 和 sys.float_info

这两个属性分别给出了 Python 中两个重要的数据类型的相关信息。

其中 `sys.int_info` 的值为：

```
>>> sys.int_info
sys.int_info(bits_per_digit=30, sizeof_digit=4)
```

在文档中的解释为：

属性解释bits_per_digitnumber of bits held in each digit. Python integers are stored internally in base `2**int_info.bits_per_digit` sizeof_digitsize in bytes of the C type used to represent a digit

指的是 Python 以 2 的 `sys.int_info.bits_per_digit` 次方为基来表示整数，也就是说它是“2 的 `sys.int_info.bits_per_digit` 次方进制”的数。这样的数每一个为都用 C 类中的 4 个字节来存储。

换句话说，每“进 1 位”（即整数值增加2 的 `sys.int_info.bits_per_digit` 次方），就需要多分配 4 个字节用以保存某个整数。

因此在 `sys.getsizeof()` 的示例中，我们可以看到 `2**30-1` 和 `2**30` 之间，虽然本身只差了 1，但是所占的字节后者却比前者多了 4。

而 `sys.float_info` 的值则是：

```
>>> sys.float_info
sys.float_info(max=1.7976931348623157e+308, max_exp=1024, max_10_exp=308, min=2.225073
```

其中各项具体的含义就不在这里继续展开了，感兴趣的同学可以参看文档和《深入理解计算机系统》等讲解组成原理的书。

4. 一个有趣的功能

接下来让我们放松一下。

每次打开 Python 的交互式界面，我们都会看到一个提示符 `>>>` 。不知道你有没有想过要把这个东西换成另外的什么呢？

反正我没想过哈哈——至少在文档中看到这两个属性之前，我确实没有想过。哪两个属性呢？

就这俩货： `sys.ps1` 和 `sys.ps2`

所谓“ps”，应当是“prompts”的简写，即“提示符”。

这两个属性中 `sys.ps1` 代表的是一级提示符，也就是进入 Python 交互界面之后就会出现的那

好了，知道怎么回事儿就好办了。

现在我们就来一个：

```
>>> sys.ps1 = "justdopython "
justdopython li = [1,2,3]
justdopython li[0]
1
justdopython
```

提示符已经被改变了，当然，有点长，不大美观哈哈。

咱们换一下：

```
justdopython sys.ps1 = "ILoveYou: "
ILoveYou: print("你可真是个小机灵鬼儿！")
你可真是个小机灵鬼儿！
ILoveYou:
```

有点儿意思吧？

注意不要忘了在字符串最后加个空格，否则提示符就会和你输入的内容混杂在一起了，会十分难看的哟~

> 示例代码：python-100-days

参考资料

Python3 标准库-sys 模块

发布于 2020-06-26 17:10

Python 3.x　Python　Python 标准库

写下你的评论...

6 条评论　默认　最新

hkt1998
第一个例子中，为什么脚本名称是sys_argv_example.py，但输出的是example.py？是打错了吗？
2022-08-22　回复　4

zero
那必须是打错了。
2023-12-25　回复　喜欢

chillout2333
网站链接是不是被dns劫持了

2021-09-13 回复 喜欢

溯流光
感谢，很明朗
2020-11-27 回复 喜欢

在下谢某人
递归好用 优秀
2022-03-14 回复 喜欢

文章被以下专栏收录

好好学习

推荐阅读

ython模块之 sys 模块

不卑不亢当仁不让

python自定义模块

模块的引入方式：1.import 模块名：导入模块中的所有内容（引入多个用逗号分隔） import random,time 2.from 模块名 import 函数名1，函数名2... 导入部分模块 （导入部分的话直接使用...

乐观勇敢坚强的老彭

Python的sys模块用法

sys模块负责程序与python解释器的交互，提供了一系列的函数和变量，用于操控python的运行时环境。1.argv: 实现从程序外部向程序传递参数在F:\Pywork下的temp.py文件编写以下代码:import sys...

渔舟唱晚

最强 Python 数据可视化库，没有之一！

今天给大家分享一篇可视化干货，介绍的是功能强大的开源 Python 绘图库 Plotly，教你如何用超简单的（甚至只要一行！）代码，绘制出更棒的图表。 我之前一直守着 matplotlib 用的原因，就是...

云原生基地